Data Structures (CSN12101)
ASSIGNMENT-4

Name : Aayansh Niranjan Date :28/01/2026
Reg_No.: 20253004

Q1. Write WAP to implement Linked List.

a) Create a linked list.
b) Insert an element at the start of the linked list.
c) Insert an element at the end of the linked list.
d) Insert an element before an existing element whose information is x in a linked list.
e) Insert an element after an existing element whose information is x in a linked list.
f) Delete the first element of the linked list.
g) Delete the last element of the linked list.
h) Delete the element whose information is x from a linked list.
i) Display the contents of the linked list.

A1. CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct Node{
  struct Node* next;
  int val;
};

struct Node* insert_at_last(struct Node* head, int data){
  struct Node* newnode=(struct Node*)malloc(sizeof(struct Node));
  newnode->next=NULL;
  newnode->val=data;
  if(head==NULL){
    return newnode;
  }
  struct Node* temp=head;
  while(temp->next) temp=temp->next;
  temp->next=newnode;
  return head;
}

struct Node* insert_before_x(struct Node* head, int data, int x){
  struct Node* newnode=(struct Node*)malloc(sizeof(struct Node));
  newnode->val=data;
  if(head==NULL){printf("Blank linked list");return head;}
  struct Node* temp=head;
  struct Node* prev;
  while(temp->val!=x && temp->next){prev=temp;temp=temp->next;}
  if(!temp->next && temp->val!=x){printf("x not found in the list");return head;}
  prev->next=newnode;
  newnode->next=temp;
  return head;
}
```

```
struct Node* insert_after_x(struct Node* head, int data, int x){
  struct Node* newnode=(struct Node*)malloc(sizeof(struct Node));
  newnode->val=data;
  if(head==NULL){printf("Blank linked list");return head;}
  struct Node* temp=head;
  while(temp->val!=x && temp->next){temp=temp->next;}
  if(!temp->next){printf("x not found in the list");return head;}
  newnode->next=temp->next;
  temp->next=newnode;
  return head;
}

struct Node* delete_head(struct Node* head){
  if(head==NULL){printf("Blank linked list");return head;}
  struct Node* ans=head->next;
  free(head);
  return ans;
}

struct Node* delete_end(struct Node* head){
  if(head==NULL){printf("Blank linked list");return head;}
  struct Node* temp=head;
  struct Node* prev;
  while(temp->next){prev=temp;temp=temp->next;}
  prev->next=NULL;
  free(temp);
  return head;
}

struct Node* delete_x(struct Node* head, int x){
  if(head==NULL){printf("Blank linked list");return head;}
  struct Node* temp=head;
  struct Node* prev;
  while(temp->val!=x && temp->next){prev=temp;temp=temp->next;}
  if(!temp->next && temp->val!=x){printf("x not found in the list");return head;}
  prev->next=temp->next;
  free(temp);
  return head;
}

void printll(struct Node* head){
  if(head==NULL){printf("Blank linked list");return;}
  struct Node* temp=head;
  while(temp){
    printf("%d ",temp->val);
    temp=temp->next;
  }
  printf("\n");
}
```

```
int main(){
  struct Node* head=NULL;
  printf("Enter the number of elements in the linked list ");
  int n;
  scanf("%d",&n);
  printf("Enter the elements of the linked list space seperated ");
  for(int i=0;i<n;i++){
    int k;
    scanf("%d",&k);
    head=insert_at_last(head,k);
  }
  printll(head);
  printf("Enter the element before which you want to insert : ");
  int x;
  scanf("%d",&x);
  printf("Enter the element which you want to insert : ");
  int k;
  scanf("%d",&k);
  head=insert_before_x(head,k,x);
  printll(head);
  printf("Enter the element after which you want to insert : ");
  scanf("%d",&x);
  printf("Enter the element which you want to insert : ");
  scanf("%d",&k);
  head=insert_after_x(head,k,x);
  printll(head);
  printf("Deleting the head : ");
  head=delete_head(head);
  printll(head);
  printf("Deleting the end : ");
  head=delete_end(head);
  printll(head);
  printf("Enter the element you want to delete : ");
  scanf("%d",&x);
  head=delete_x(head,x);
  printll(head);
  return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_4$ ./p1
Enter the number of elements in the linked list 5
Enter the elements of the linked list space seperated 7 2 9 4 3
7 2 9 4 3
Enter the element before which you want to insert : 9
Enter the element which you want to insert : 3
7 2 3 9 4 3
Enter the element after which you want to insert : 2
Enter the element which you want to insert : 8
7 2 8 3 9 4 3
Deleting the head : 2 8 3 9 4 3
Deleting the end : 2 8 3 9 4
Enter the element you want to delete : 8
2 3 9 4
```

Q2. In a college library, you are tasked with implementing a feature to manage a list of books represented as a linked list, where each node contains details of a book (like title, author) and a reference to the next book. You need to write a recursive function, that takes the head of the linked list as input and returns the total number of books (i.e., the length of the linked list). The function should use recursion to traverse the list and count the number of nodes, returning 0 when the end of the list (null or none) is reached.

A2. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Book {
    char title[50];
    char author[50];
    struct Book* next;
};
struct Book* insertBook(struct Book* head, char title[], char author[]) {
    struct Book* newBook=(struct Book*)malloc(sizeof(struct Book));
    strcpy(newBook->title,title);
    strcpy(newBook->author,author);
    newBook->next=NULL;
    if (head==NULL) return newBook;
    struct Book* temp=head;
    while (temp->next) temp=temp->next;
    temp->next=newBook;
    return head;
}
int countBooks(struct Book* head) {
    if (head==NULL) return 0;
    return 1+countBooks(head->next);
}
int main() {
    struct Book* head = NULL;
    head = insertBook(head, "C Programming", "Dennis Ritchie");
    head = insertBook(head, "Data Structures", "Seymour Lipschutz");
    head = insertBook(head, "Algorithms", "Thomas Cormen");
    int total = countBooks(head);
    printf("Total number of books = %d\n", total);
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_4$ ./p2
Total number of books = 3
```

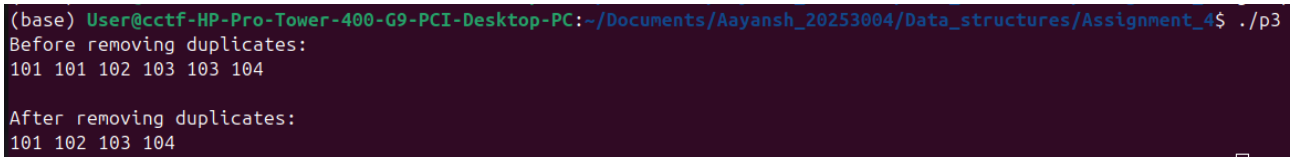
Q3. In a bank there are customers, you have a sorted linked list storing customer IDs. Sometimes, due to data entry errors, duplicate customer IDs are added to the list. You need to implement a function to remove any duplicate customer IDs from the sorted linked list, ensuring each ID appears only once.

A3. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node{
   int customerID;
   struct Node* next;
};
struct Node* insert_end(struct Node* head,int id){
   struct Node* newnode=(struct Node*)malloc(sizeof(struct Node));
   newnode->customerID=id;
   newnode->next=NULL;
   if(head==NULL)return newnode;
   struct Node* temp=head;
   while (temp->next)temp=temp->next;
   temp->next=newnode;
   return head;
}
void removeDuplicates(struct Node* head){
   if(head==NULL) return;
   struct Node* current=head;
   while(current->next!=NULL){
      if(current->customerID==current->next->customerID){
         struct Node* duplicate=current->next;
         current->next=duplicate->next;
         free(duplicate);
      }
      else current=current->next;
   }
}
void display(struct Node* head){
   struct Node* temp=head;
   while(temp){
      printf("%d ",temp->customerID);
      temp=temp->next;
   }
   printf("\n");
}
int main(){
   struct Node* head=NULL;
   head = insert_end(head,101);
   head = insert_end(head,101);
   head = insert_end(head,102);
   head = insert_end(head,103);
   head = insert_end(head,103);
   head = insert_end(head,104);
   printf("Before removing duplicates:\n");
   display(head);
   removeDuplicates(head);
   printf("\nAfter removing duplicates:\n");
   display(head);
}
```

OUTPUT:


(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_4$ ./p3
Before removing duplicates:
101 101 102 103 103 104
After removing duplicates:
101 102 103 104

Q4. In an inventory management system, you have two separate linked lists representing different categories of products. You need to merge these two linked lists into one, maintaining the order of products in both lists. Write a function concatenate_lists(list1, list2) to concatenate the two linked lists. Ensure the products from both categories are combined into a single linked list.
A4. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int productID;
   struct Node* next;
};
struct Node* insert_end(struct Node* head, int id) {
   struct Node* newnode = (struct Node*)malloc(sizeof(struct Node));
   newnode->productID = id;
   newnode->next = NULL;
   if (head==NULL) return newnode;
   struct Node* temp=head;
   while (temp->next)
      temp=temp->next;
   temp->next=newnode;
   return head;
}
struct Node* concatenate_lists(struct Node* list1, struct Node* list2) {
   if(list1==NULL)return list2;
   struct Node* temp=list1;
   while (temp->next)temp=temp->next;
   temp->next=list2;
   return list1;
}
void display(struct Node* head) {
   struct Node* temp = head;
   while(temp){
      printf("%d ",temp->productID);
      temp=temp->next;
   }
   printf("\n");
}
int main() {
   struct Node* list1=NULL;
   struct Node* list2=NULL;
   list1 = insert_end(list1,1);
   list1 = insert_end(list1,2);
   list1 = insert_end(list1,3);
   list2 = insert_end(list2,4);
   list2 = insert_end(list2,5);
   printf("Category 1 Products:\n");
   display(list1);
   printf("Category 2 Products:\n");
   display(list2);
   list1 = concatenate_lists(list1, list2);
   printf("\nCombined Product List:\n");
   display(list1);
   return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_4$ ./p4
Category 1 Products:
1 2 3
Category 2 Products:
4 5

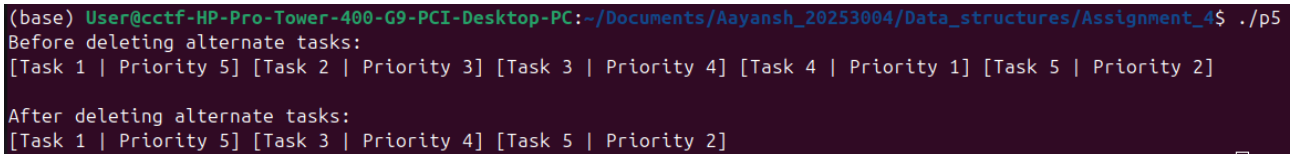
Combined Product List:
1 2 3 4 5
```

Q5. In a task scheduling system, you maintain a linked list of tasks, where each node represents a task and its priority. To optimize the schedule, you need to remove every alternate task from the list, effectively deleting tasks that are not critical. Write a function delete_alternate_nodes(head) to delete every alternate node in the linked list. The function should update the list by removing every second task. Ensure the solution works for any size of the linked list.
A5. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int taskID;
   int priority;
   struct Node* next;
};
struct Node* insert_end(struct Node* head, int id, int priority) {
   struct Node* newnode=(struct Node*)malloc(sizeof(struct Node));
   newnode->taskID=id;
   newnode->priority=priority;
   newnode->next=NULL;
   if (head==NULL) return newnode;
   struct Node* temp=head;
   while(temp->next)temp=temp->next;
   temp->next=newnode;
   return head;
}
void delete_alternate_nodes(struct Node* head) {
   if(head==NULL) return;
   struct Node* current=head;
   while(current && current->next){
      struct Node* temp=current->next;
      current->next=temp->next;
      free(temp);
      current=current->next;
   }
}
void display(struct Node* head) {
   struct Node* temp=head;
   while (temp){
      printf("[Task %d | Priority %d] ",temp->taskID,temp->priority);
      temp=temp->next;
   }
   printf("\n");
}
int main(){
   struct Node* head=NULL;
   head = insert_end(head,1,5);
   head = insert_end(head,2,3);
   head = insert_end(head,3,4);
   head = insert_end(head,4,1);
   head = insert_end(head,5,2);
   printf("Before deleting alternate tasks:\n");
   display(head);
   delete_alternate_nodes(head);
   printf("\nAfter deleting alternate tasks:\n");
   display(head);
   return 0;
}
```

OUTPUT:


(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_4$ ./p5
Before deleting alternate tasks:
[Task 1 | Priority 5] [Task 2 | Priority 3] [Task 3 | Priority 4] [Task 4 | Priority 1] [Task 5 | Priority 2]
After deleting alternate tasks:
[Task 1 | Priority 5] [Task 3 | Priority 4] [Task 5 | Priority 2]

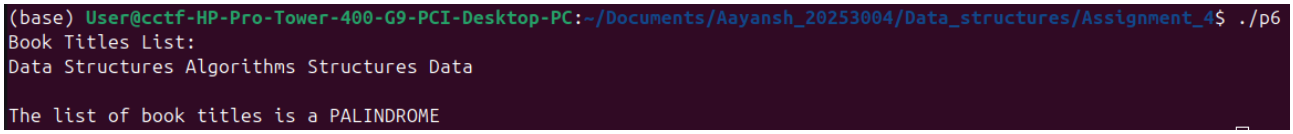
Q6. In a document management system, you store the titles of books in a linked list, with each node containing a book title. You need to check if the list of book titles forms a palindrome, meaning the titles are the same when read forwards and backwards. Write a function to determine if the linked list of book titles is a palindrome. The function should return true if the titles are the same in both directions, and False otherwise. Ensure that the solution works with any length of linked list.
A6. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char title[50];
    struct Node* next;
};
struct Node* insert_end(struct Node* head, char title[]) {
    struct Node* newnode=(struct Node*)malloc(sizeof(struct Node));
    strcpy(newnode->title,title);
    newnode->next=NULL;
    if (head==NULL) return newnode;
    struct Node* temp=head;
    while(temp->next)temp=temp->next;
    temp->next=newnode;
    return head;
}
int isPalindrome(struct Node* head) {
    char titles[100][50];
    int count = 0;
    struct Node* temp=head;
    while(temp!=NULL){
        strcpy(titles[count],temp->title);
        count++;
        temp=temp->next;
    }
    int start=0,end=count-1;
    while(start<end){
        if(strcmp(titles[start],titles[end])!=0) return 0;
        start++;
        end--;
    }
    return 1;
}
void display(struct Node* head){
    struct Node* temp=head;
    while(temp){
        printf("%s ",temp->title);
        temp = temp->next;
    }
    printf("\n");
}
```

```
int main(){
    struct Node* head=NULL;
    head=insert_end(head,"Data");
    head=insert_end(head,"Structures");
    head=insert_end(head,"Algorithms");
    head=insert_end(head,"Structures");
    head=insert_end(head,"Data");
    printf("Book Titles List:\n");
    display(head);
    if (isPalindrome(head)) printf("\nThe list of book titles is a PALINDROME\n");
    else printf("\nThe list of book titles is NOT a PALINDROME\n");
    return 0;
}
```

OUTPUT:



Q7. In a task management system, you store a list of tasks in a linked list, with each node representing a task and its priority. You want to reverse the order of tasks, so that the last task added appears first and the first task appears last. Write a function that should rearrange the nodes in reverse order without creating a new list. Make sure the solution works efficiently for any number of tasks.

A7. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int taskID;
   int priority;
   struct Node* next;
};
struct Node* insert_end(struct Node* head, int id, int priority){
   struct Node* newnode=(struct Node*)malloc(sizeof(struct Node));
   newnode->taskID=id;
   newnode->priority=priority;
   newnode->next=NULL;
   if(head==NULL) return newnode;
   struct Node* temp=head;
   while(temp->next) temp=temp->next;
   temp->next=newnode;
   return head;
}
struct Node* reverse_list(struct Node* head) {
   struct Node* prev=NULL;
   struct Node* curr=head;
   struct Node* next=NULL;
   while(curr){
      next=curr->next;
      curr->next=prev;
      prev=curr;
      curr=next;
   }
   return prev;
}
void display(struct Node* head) {
   struct Node* temp=head;
   while(temp){
      printf("[Task %d | Priority %d] ",temp->taskID,temp->priority);
      temp=temp->next;
   }
   printf("\n");
}
int main() {
   struct Node* head=NULL;
   head=insert_end(head,1,5);
   head=insert_end(head,2,3);
   head=insert_end(head,3,4);
   head=insert_end(head,4,1);
   printf("Original Task Order:\n");
   display(head);
   head=reverse_list(head);
   printf("\nReversed Task Order:\n");
   display(head);
   return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_4$ ./p7
Original Task Order:
[Task 1 | Priority 5] [Task 2 | Priority 3] [Task 3 | Priority 4] [Task 4 | Priority 1]

Reversed Task Order:
[Task 4 | Priority 1] [Task 3 | Priority 4] [Task 2 | Priority 3] [Task 1 | Priority 5]
```